Computer System Design & Application

计算机系统设计与应用A

陶伊达 （TAO Yida）
taoyd@sustech.edu.cn

Lecture 2

- Generics
- Collections

What is Generics?

Why do we need Generics?

A World without Generics

```
public class ArrayList {

    private Object[] elements;
    ……
    public Object get(int i){…}
    public void add(Object o){…}
}
```

A World without Generics

Drawback 1: Explicit casting is required; inefficient and hard to read

```
ArrayList list = new ArrayList();
list.add("hi");
String s = list.get(0);
```

❌

Compiler error:
Type mismatch: cannot convert from Object to String

Need explicitly cast to String

```
String s = (String)list.get(0);
```

A World without Generics

Drawback 2: error-prone; may cause type-related runtime errors if a programmer makes a mistake with the explicit casting.

No compilation error here
(no error checking)

```
ArrayList list = new ArrayList();
list.add("Hello");
list.add(2022);

for(int i=0;i<list.size();i++) {
    String elem = (String)list.get(i);
    System.out.println(elem);
}
```

But here throws ClassCastException:
class java.lang.Integer cannot be cast to
class java.lang.String

Solution?

What's the problem with this solution?

- Using a dedicated list for each type
  - StringArrayList
  - IntegerArrayList
  - CharArrayList
  - BoolArrayList
  - ……
- Infeasible solution
  - Too many kinds of list (thousands in Java)
  - Too much duplication
  - Hard to scale for user-defined objects

Solution: Generics

- Introduced in JDK 5.0
- Parameterized types: types like classes and interfaces can be used as parameters

```
public class ArrayList<E>
```

```
public boolean add(E e)
```

Appends the specified element to the end of this list.

```
public E get(int index)
```

Returns the element at the specified position in this list.

- E stands for “element” (sometimes we use T)
- E could be any **non-primitive** type
- All elements of the list should be of type E

Solution: Generics

```
// Code is easier to read
// You can tell right away that this list contains String
ArrayList<String> list = new ArrayList<String>();
list.add("Hello");

// Compiler checks that you don't insert object
// of the wrong type
list.add(2022);  ✗

// No explicit cast is required
// Compiler will add the correct type cast
String elem = list.get(0);
```

Comparisons

It's better to discover errors as early as possible!

Could put anything into the list; compiler won't complain

Need explicit type cast to get element; prone to runtime errors (crash)

Could only put the specified element; otherwise compiler will complain

No need for type cast since type-safety is already guaranteed in compile time

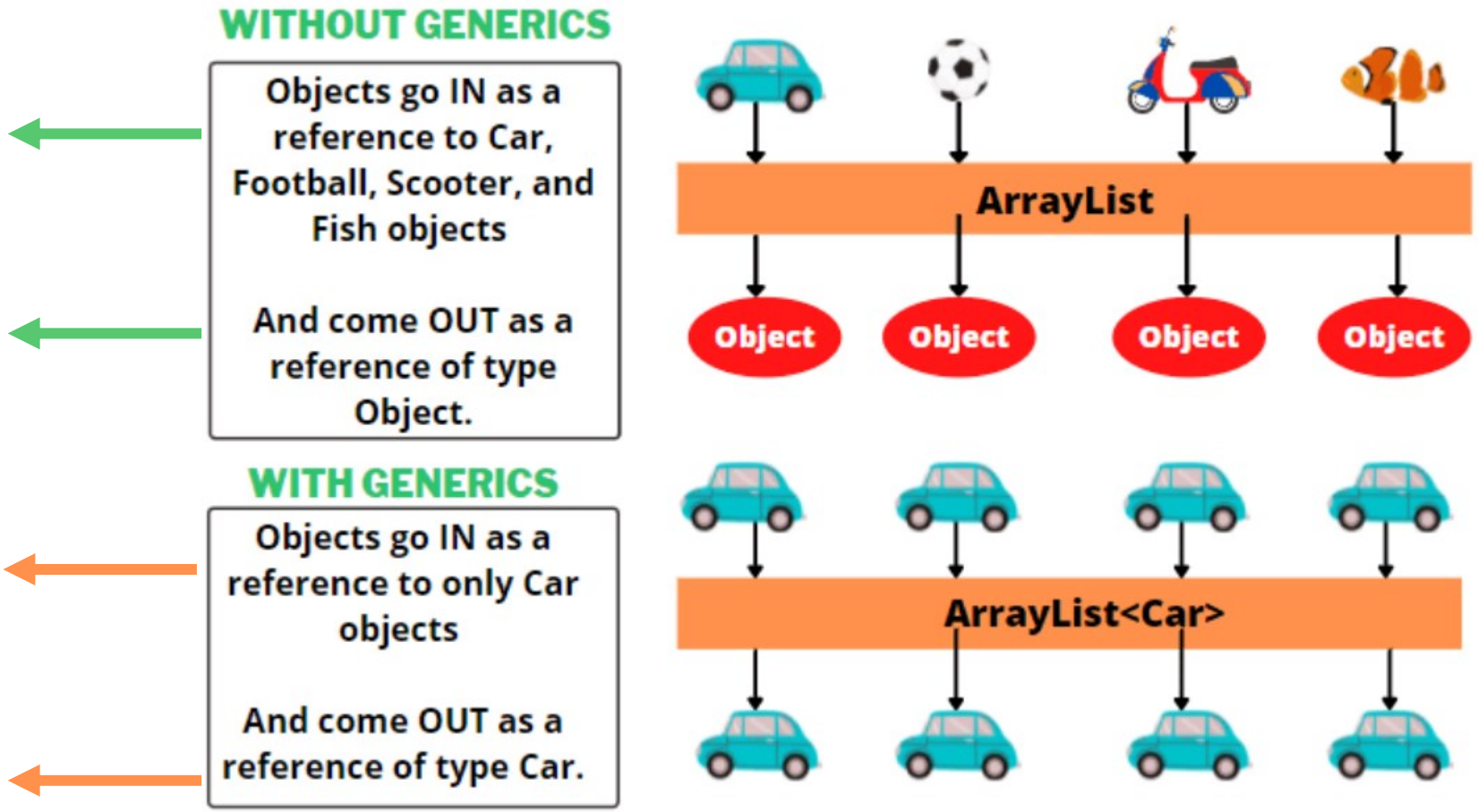


Image source: https://www.scientecheasy.com/2021/10/generics-in-java.html/

Terms

Example	Term
List<E>	Generic type
E	Formal type parameter (类型形参) Type variable
List<String>	Parameterized type
String	Actual type parameter (类型实参)
List	Raw type (原始类型)

(Avoid) Using Raw Types

```
List list = new ArrayList();
```

ArrayList is a raw type. References to generic type ArrayList<E> should be parameterized

By using raw types, we'll lose all the type-safety and expressiveness benefits of generics

Question: but the code could still compile (warning instead of error) and run, why?

Using Generics

- Generic classes
- Generic interfaces
- Generic methods

Classes in Java Collections (e.g., List, Queue, Set) are typically generic classes

```
/**
 * @version 1.00 2004-05-10
 * @author Cay Horstmann
 */
public class Pair<T>
{
   private T first;
   private T second;

   public Pair() { first = null; second = null; }

   public Pair(T first, T second) {
      this.first = first;  this.second = second;
   }


   public T getFirst() { return first; }
   public T getSecond() { return second; }

   public void setFirst(T newValue) { first = newValue; }
   public void setSecond(T newValue) { second = newValue; }
}
```

Using Generics

- Generic classes
- Generic interfaces
- Generic methods

```
public interface Comparable<T>
```

Prior to JDK 1.5 (and Generic Types):

```
public interface Comparable {
   public int compareTo(Object o) }

Comparable c = new Date();
System.out.println(c.compareTo("red"));
```

run-time error

JDK 1.5 (Generic Types):

```
public Interface Comparable<T> {
   public int compareTo(T o) }

Comparable<Date> c = new Date();
System.out.println(c.compareTo("red"));
```

compile-time error

Image source: https://www.cs.rit.edu/~rlaz/cs2/slides/CS2_Week5.pdf

Using Generics

- Generic classes
- Generic interfaces
- Generic methods: Methods that introduce their own type parameters

```
// Generic method
public static <E> Set<E> union(Set<E> s1, Set<E> s2) {
    Set<E> result = new HashSet<>(s1);
    result.addAll(s2);
    return result;
}
```

Example from “Effective Java”

You can define generic methods both inside ordinary classes and inside generic classes

Bounds for Type Variables

```
<T extends BoundingType>
```

- T could be any subtype of the bounding type
- Both T and bounding type can be either a class or an interface
- Multiple bounds are allowed, separated by & (a class must be the first one in the bounds list)

```
<T extends Animal & Comparable>
```

Bounds for Type Variables

```
public static <T extends Comparable> Pair<T> minmax(T[] a)
{
   if (a == null || a.length == 0) return null;
   T min = a[0];
   T max = a[0];
   for (int i = 1; i < a.length; i++)
   {
      if (min.compareTo(a[i]) > 0) min = a[i];
      if (max.compareTo(a[i]) < 0) max = a[i];
   }
   return new Pair<>(min, max);
}
```

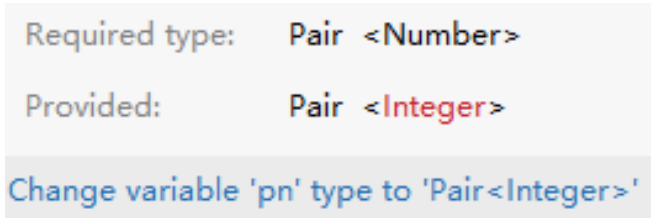
```
min = 2
max = is
min = 1815-12-10
max = 1910-06-22
```

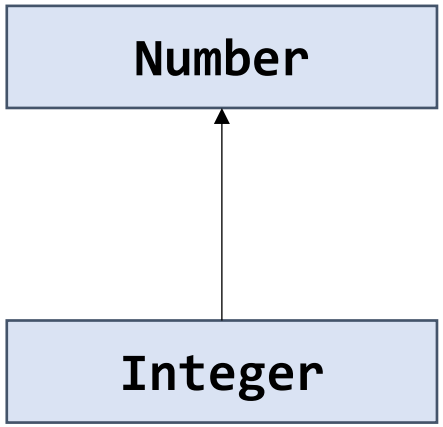
```
String[] words = {"This", "is", "CS209a", "Java", "2"};
Pair<String> mm1 = minmax(words);
System.out.println("min = " + mm1.getFirst());
System.out.println("max = " + mm1.getSecond());

LocalDate[] birthdays =
{
      LocalDate.of(1906, 12, 9), // G. Hopper
      LocalDate.of(1815, 12, 10), // A. Lovelace
      LocalDate.of(1903, 12, 3), // J. von Neumann
      LocalDate.of(1910, 6, 22), // K. Zuse
};
Pair<LocalDate> mm2 = minmax(birthdays);
System.out.println("min = " + mm2.getFirst());
System.out.println("max = " + mm2.getSecond());
```

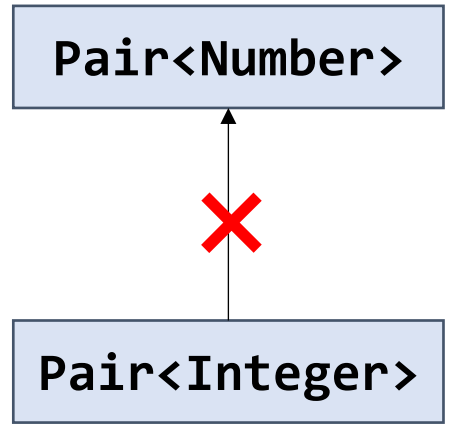
Example adapted from “Core Java Volume II”

Inheritance Rules for Generic Types

```
Pair<Integer> pi = new Pair<>(1, 2);

Pair<Number>  pn = pi;
```

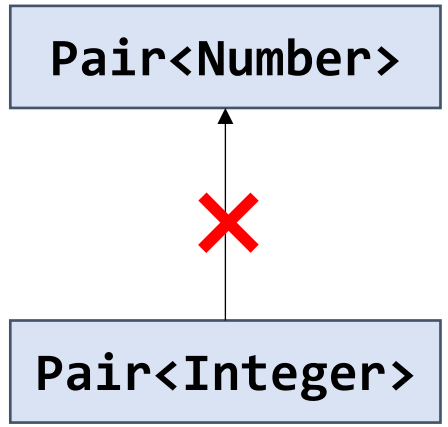






Inheritance Rules for Generic Types

```
Pair<Integer> pi = new Pair<>(1, 2);
Pair<Number> pn = pi;   // suppose this was allowed

pn.setFirst(3.14);      // legal, since pn is Pair<Number>
                        // and Double is a Number

Integer x = pi.getFirst(); // boom! now pi contains a Double,
                           // but we expect Integer
```



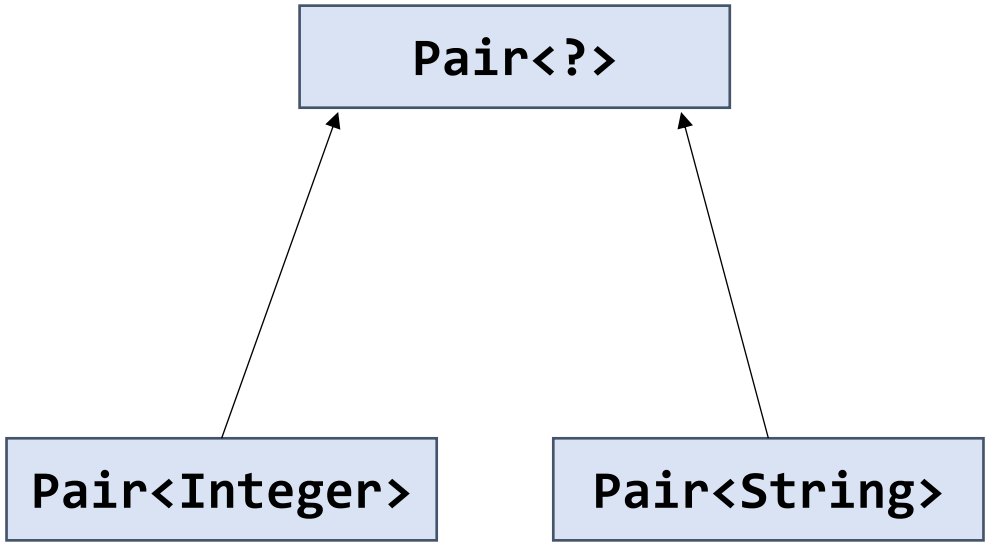
Type safety will break, so Java forbids it.

Wildcards (通配符)

Unbounded:
`Pair<?>` is a superclass of `Pair<T>` for any `T`

```
Pair<Integer> pi = new Pair<>(1,2);
Pair<?> pn = pi;
```

```
Pair<String> ps = new Pair<>(“Hi”,“there”);
Pair<?> pn = ps;
```

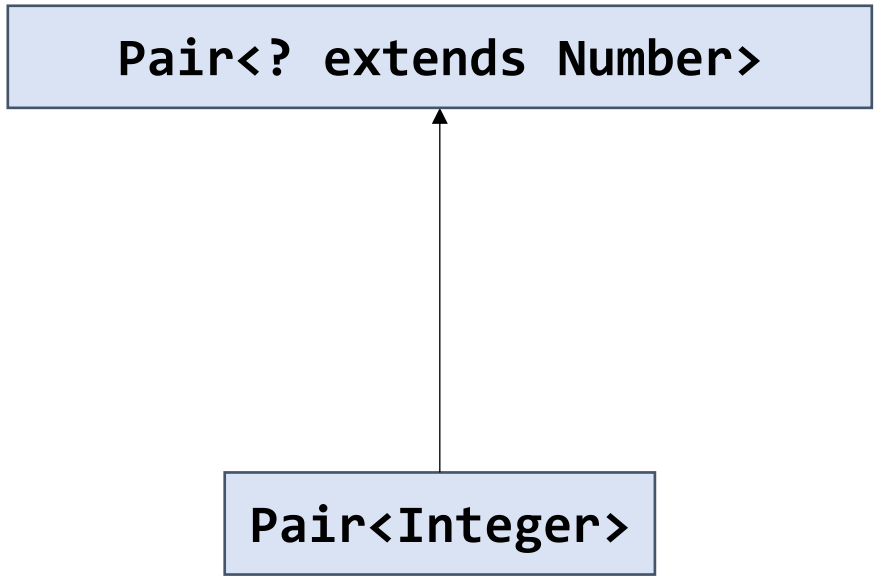


Wildcards (通配符)

Upper bounded (Bounded by the superclass):
`Pair<? extends T>`: a pair of any type that is a subtype of `T`

```
Pair<Integer> pi = new Pair<>(1,2);
Pair<? extends Number> pn = pi;
```

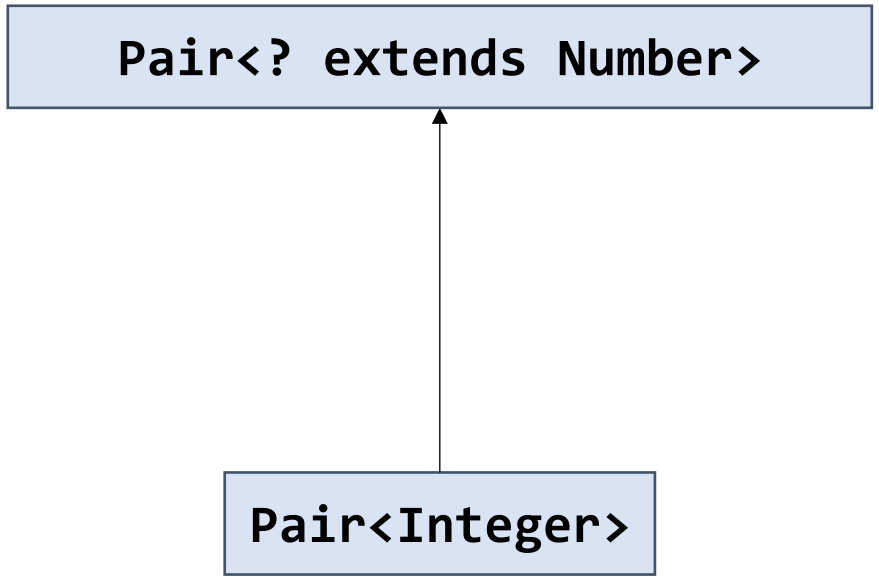
```
Pair<String> ps = new Pair<>("Hi","there");
Pair<? extends Number> pn = ps;
```

Wildcards (通配符)

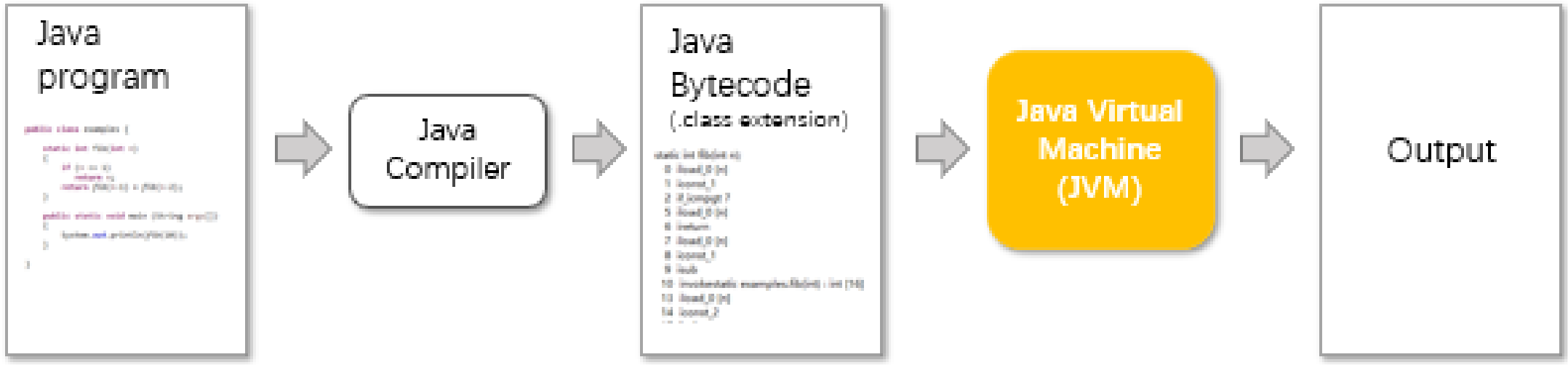
Upper bounded (Bounded by the superclass):
`Pair<? extends T>`: a pair of any type that is a subtype of `T`

```
Pair<Integer> pi = new Pair<>(1,2);
Pair<? extends Number> pn = pi;

pn.setFirst(3);      // Can we do this?
```



Design choice of Java Generics

- Generics was introduced in JDK 5.0
- What should be updated in JDK?



Type Erasure

- To be compatible with previous versions, the implementation of Java generics adopts the strategy of pseudo generics
- Java supports generics in syntax, but the so-called "type erase" will be carried out in the compilation stage to replace all generic representations with specific types
- To JVM, there is no generics at all

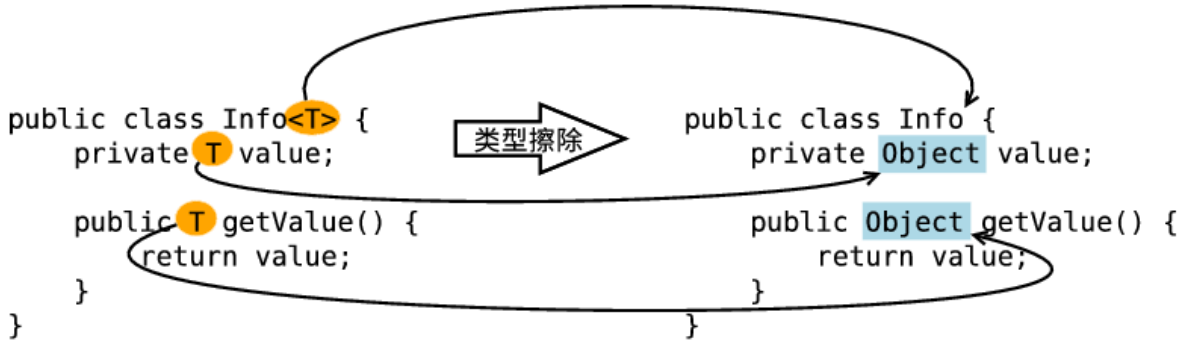


Figure 1: erasing type parameters in class definitions

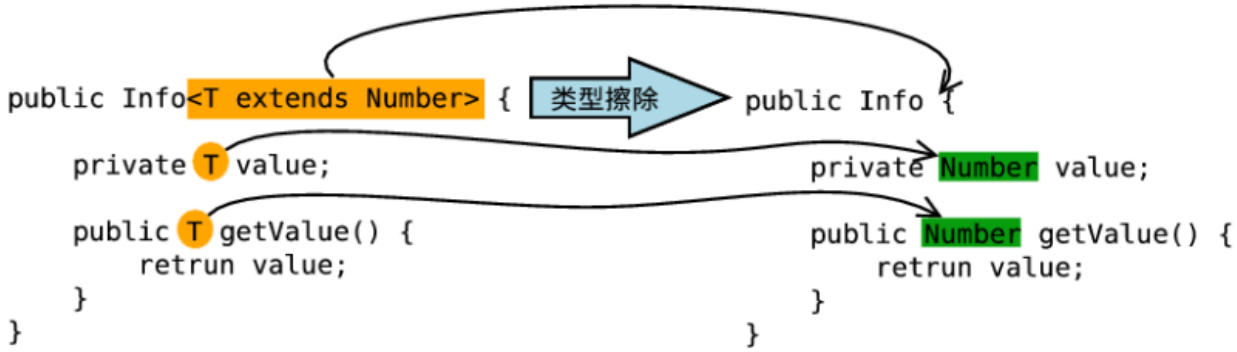


Figure 2: restricted type parameters in erase class definition

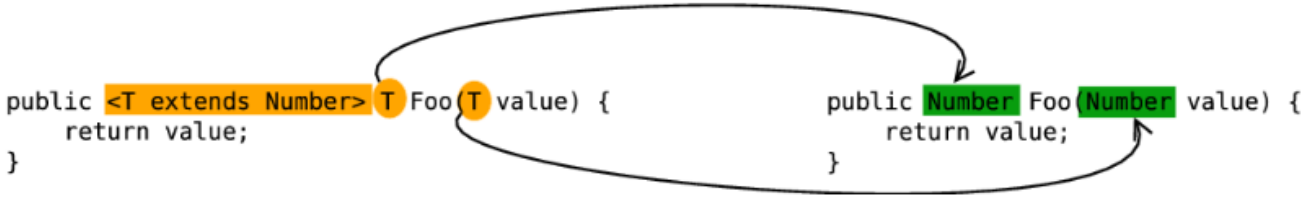


Figure 3: erasing type parameters in generic methods

https://developpaper.com/detailed-explanation-of-type-erasure-examples-of-java-generics/

Limitations of Java Generics/Type Erasure

Further Reading:

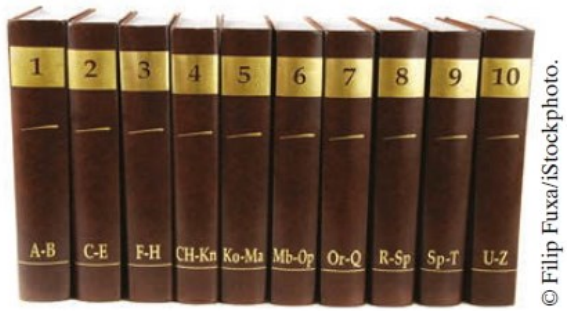
- Chapter 8.6. Restrictions and limitations. Core Java Volume 1 – Fundamentals. 11th Edition. Cay S. Horstmann
- Chapter 5: Generics. Effective Java. 3rd Edition. Joshua Bloch.
- 第10章 10.3.1 泛型. 深入理解Java虚拟机：JVM高级特性与最佳实践 (第三版). 周志明

Lecture 2

- Generics
- Collections

Concepts of Collections List, Stack, Map and Set?

A list is a collection that remembers the order of its elements.



© Filip Fuxa/iStockphoto.

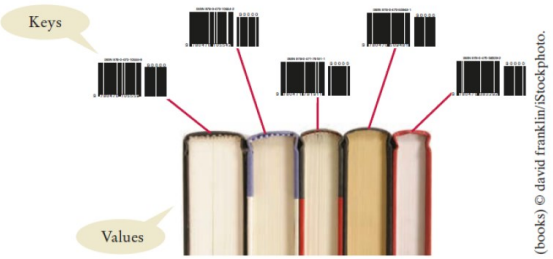
A set is an unordered collection of unique elements.


© parema/iStockphoto.

A stack is a collection of elements with "last-in, first-out" retrieval.

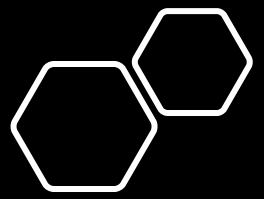

© Vladimir Trenin/iStockphoto.

A map keeps associations between key and value objects.



(books) © david franklin/iStockphoto.

Materials from the slides of Dr. HE Mingxin

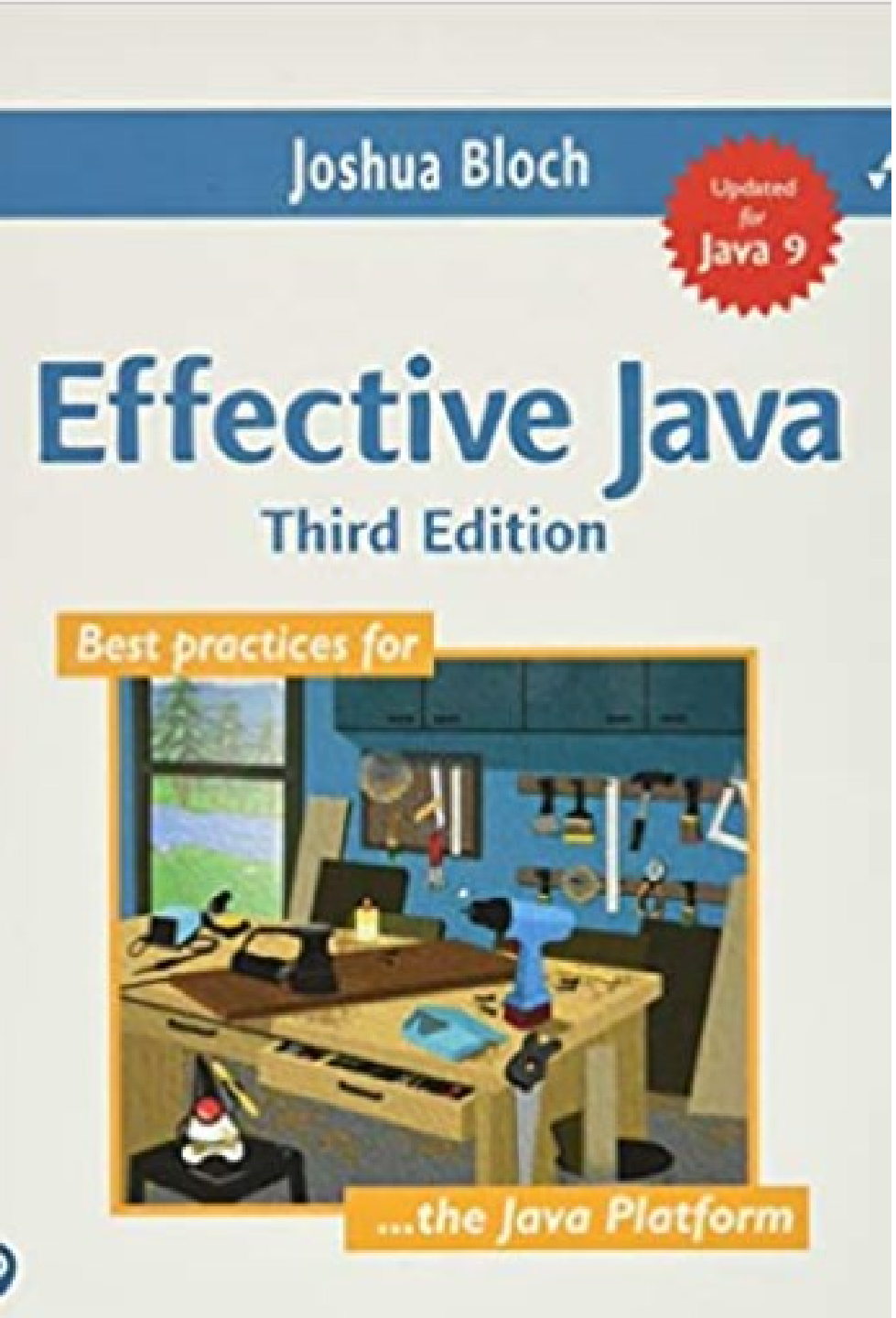


The Java Collections Framework

- Collection
  - A group of objects
  - Mainly used for data storage, data retrieval, and data manipulation
- Framework
  - A set of classes and interfaces which provide a ready-made architecture.
- Collections Framework
  - A unified architecture for representing and manipulating collections
  - Reusable data structures & functionalities
  - Collections can be manipulated independently of the details of their implementations

History

- Before JDK 1.2 (’90s)
  - Java only has `Arrays`, `Vectors`, and `Hashtables` for grouping objects
  - They are defined independently with no common interface (although many concepts are the same)
  - Difficult to use, to remember, and to extend
- The Collections Framework was introduced in JDK 1.2 (1998)
  - Consistent APIs for common functionalities (e.g., add())
  - Reducing programming & design efforts
  - Increases program speed and quality



The collections framework was designed and developed primarily by Joshua Bloch

The Java™ Platform Collections Framework

Joshua Bloch
Sr. Staff Engineer, Collections Architect
Sun Microsystems, Inc.

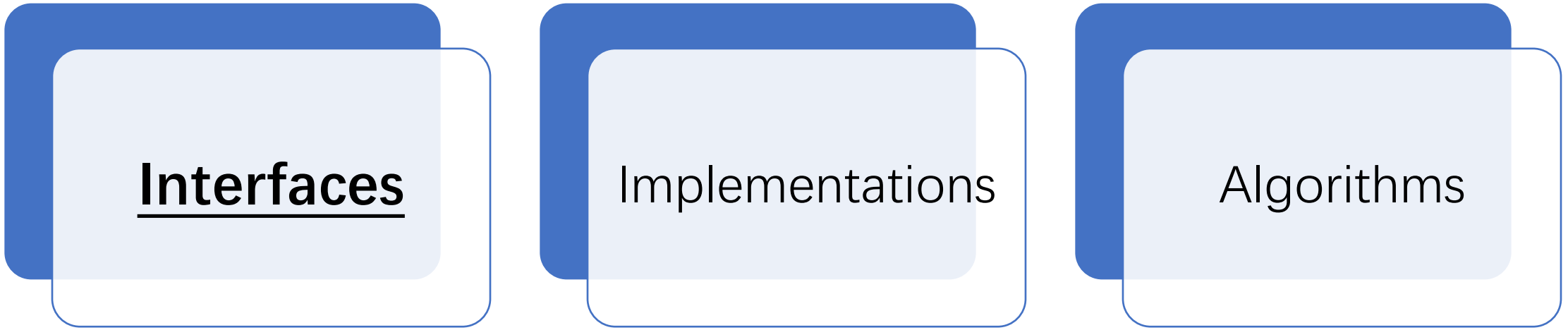
School of Computer Science
isr institute for SOFTWARE RESEARCH

JAVA

15-214 5 isr institute for SOFTWARE RESEARCH

(https://www.cs.cmu.edu/~charlie/courses/15-214/2016-fall/slides/15-collections%20design.pdf)

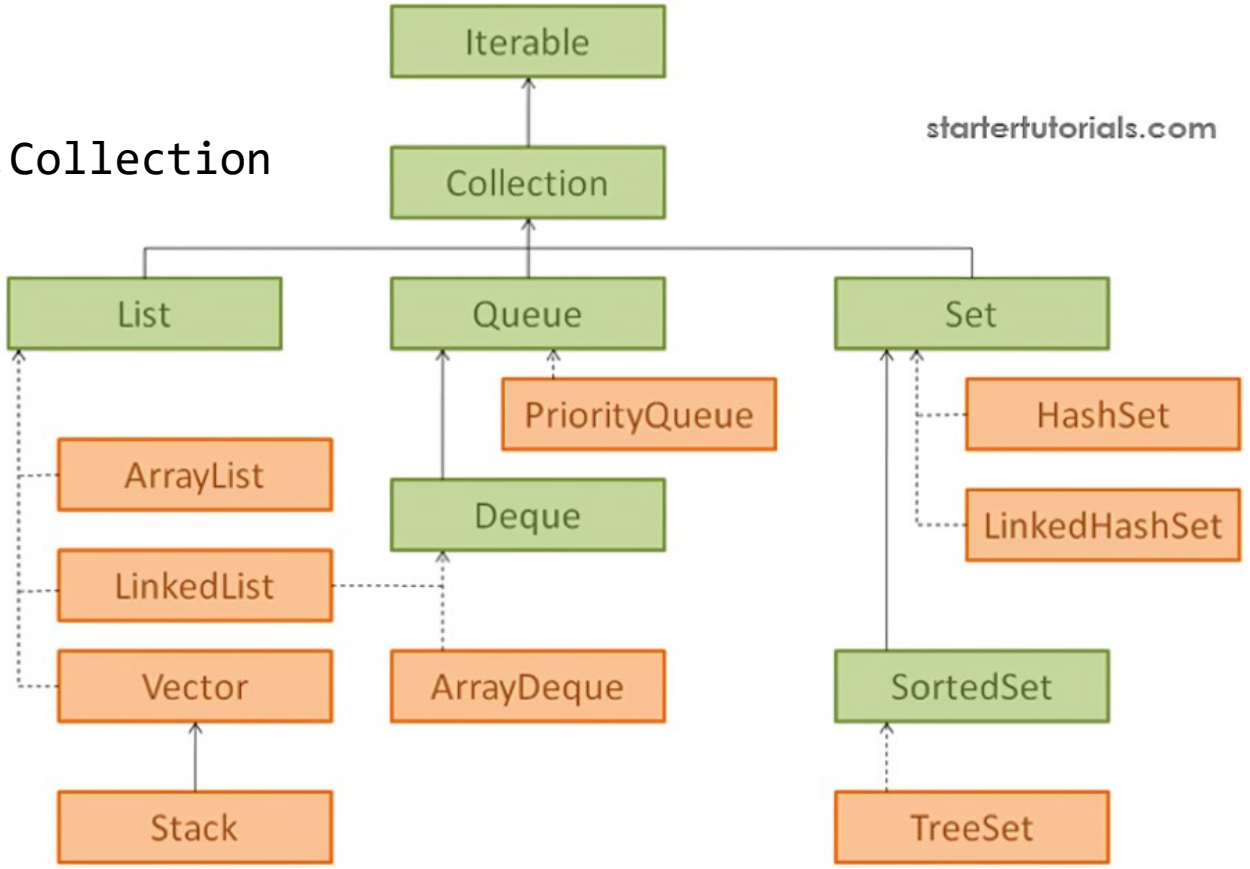
Core Elements in the Java Collections Framework



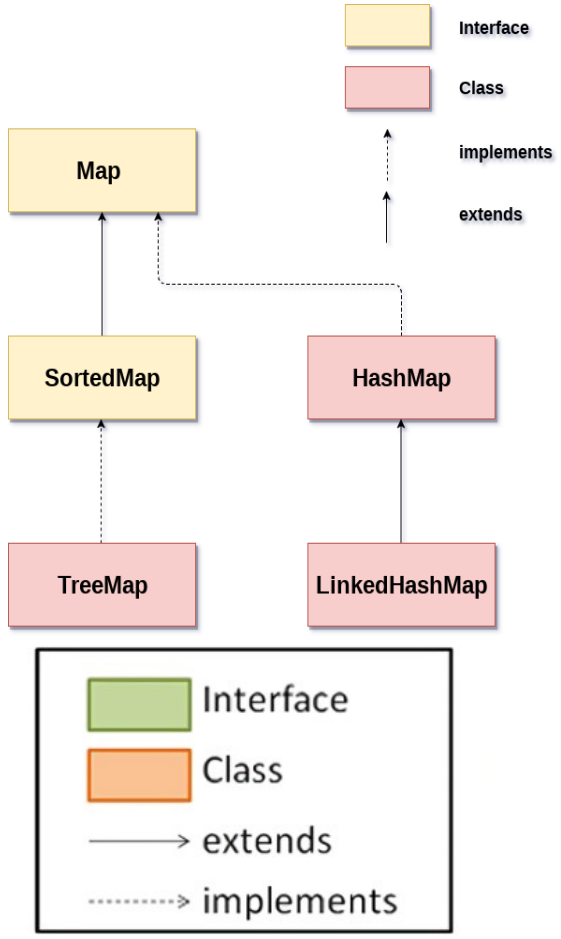
Parts of the following materials are adapted from the original slides from Josh Bloch

Collection Class Hierarchy

`java.util.Collection`



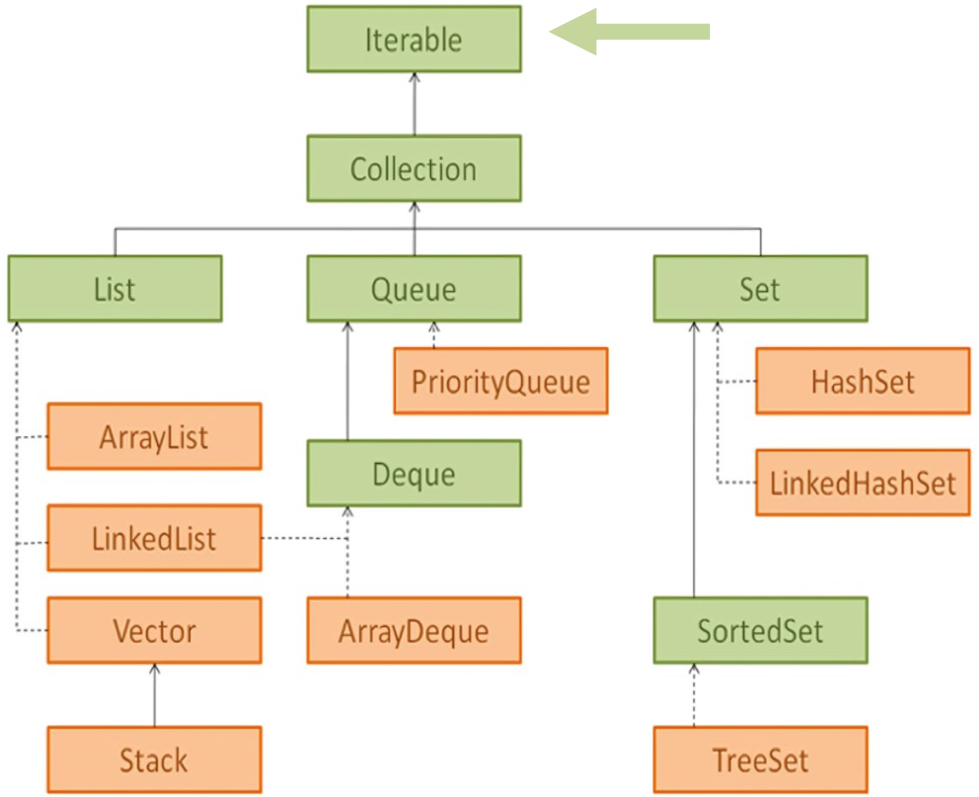
`java.util.Map`



The `Iterable<T>` interface

- Iterable：可迭代的、可遍历的
- Implementing this interface allows an object to be the iterated (with enhanced for loop)

```
for (String element : c)
{
    do something with element
}
```

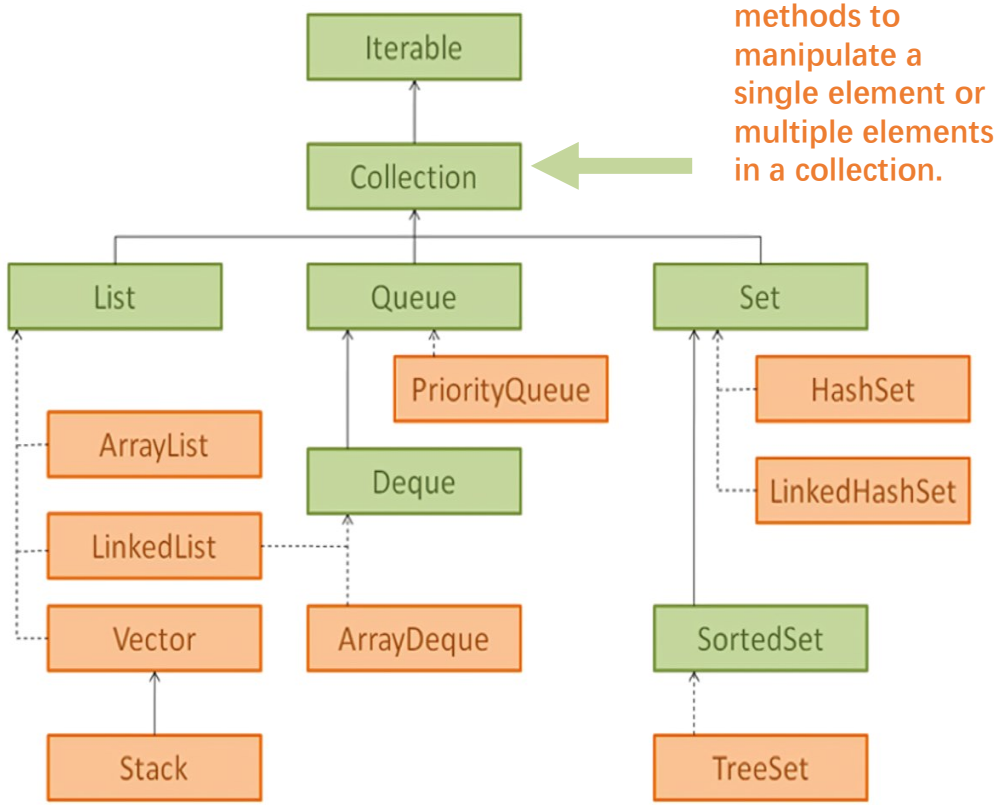


Collection Interface

```
public interface Collection<E> {
    int size();
    boolean isEmpty();
    boolean contains(Object element);
    boolean add(E element);          // Optional
    boolean remove(Object element); // Optional
    Iterator<E> iterator();

    Object[] toArray();
    T[] toArray(T a[]);

    // Bulk Operations
    boolean containsAll(Collection<?> c);
    boolean addAll(Collection<? Extends E> c); // Optional
    boolean removeAll(Collection<?> c);  // Optional
    boolean retainAll(Collection<?> c);  // Optional
    void clear();                        // Optional
}
```

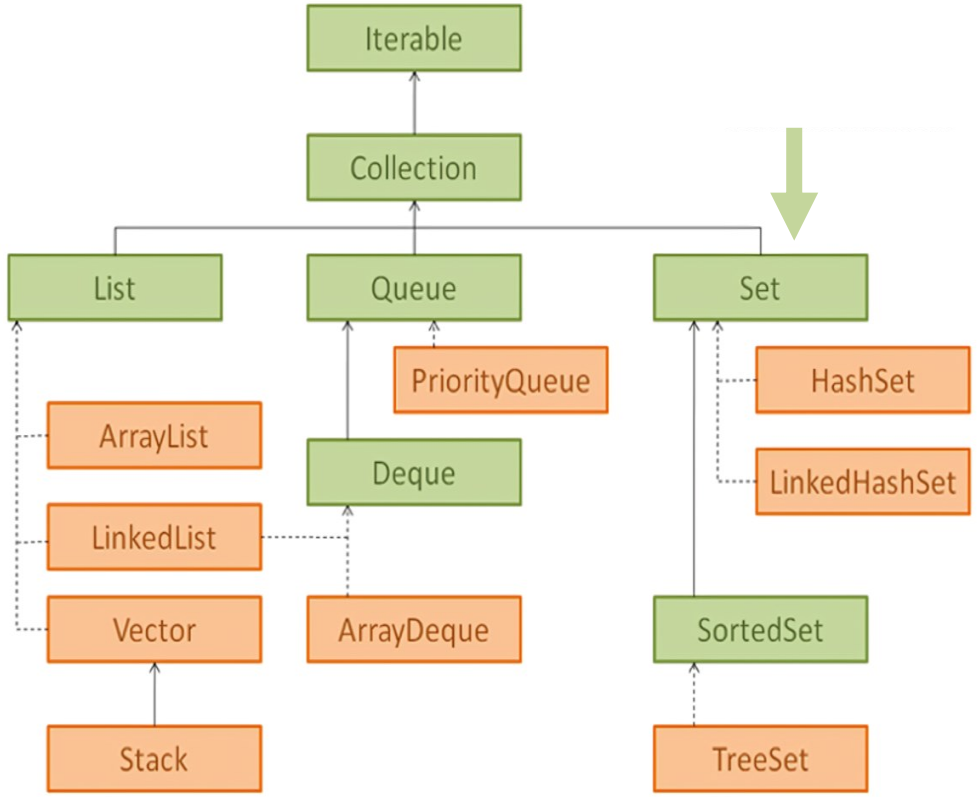
```
public interface Collection<E>
extends Iterable<E>
```

This interface has common methods to manipulate a single element or multiple elements in a collection.



`Set` Interface

- Add no new methods to `Collection`
- Override the `add()` behavior to ensure **no duplication**
- Override the `equals()` and `hashCode()` behavior to calculate **set equality**



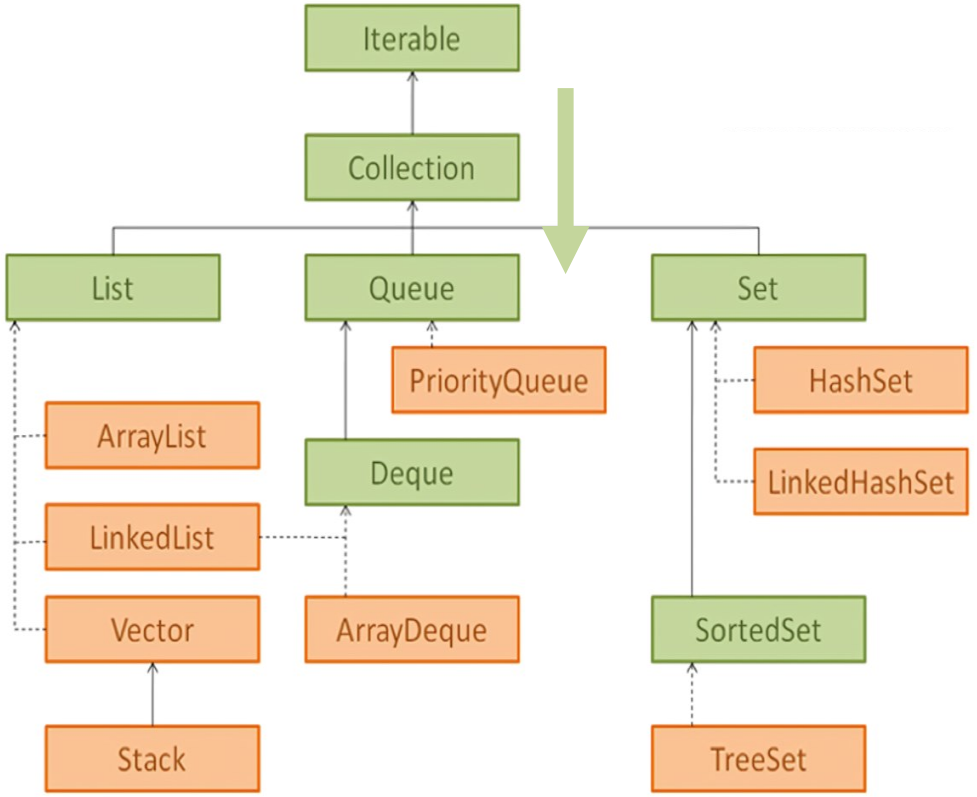
Set Idioms

```
Set<Type> s1, s2;

boolean isSubset = s1.containsAll(s2);

Set<Type> union = new HashSet<>(s1);
union = union.addAll(s2);

Set<Type> intersection = new HashSet<>(s1);
intersection.retainAll(s2);

Set<Type> difference = new HashSet<>(s1);
difference.removeAll(s2);
```

Common behaviors (containsAll, addAll, retainAll, removeAll) from Collections!

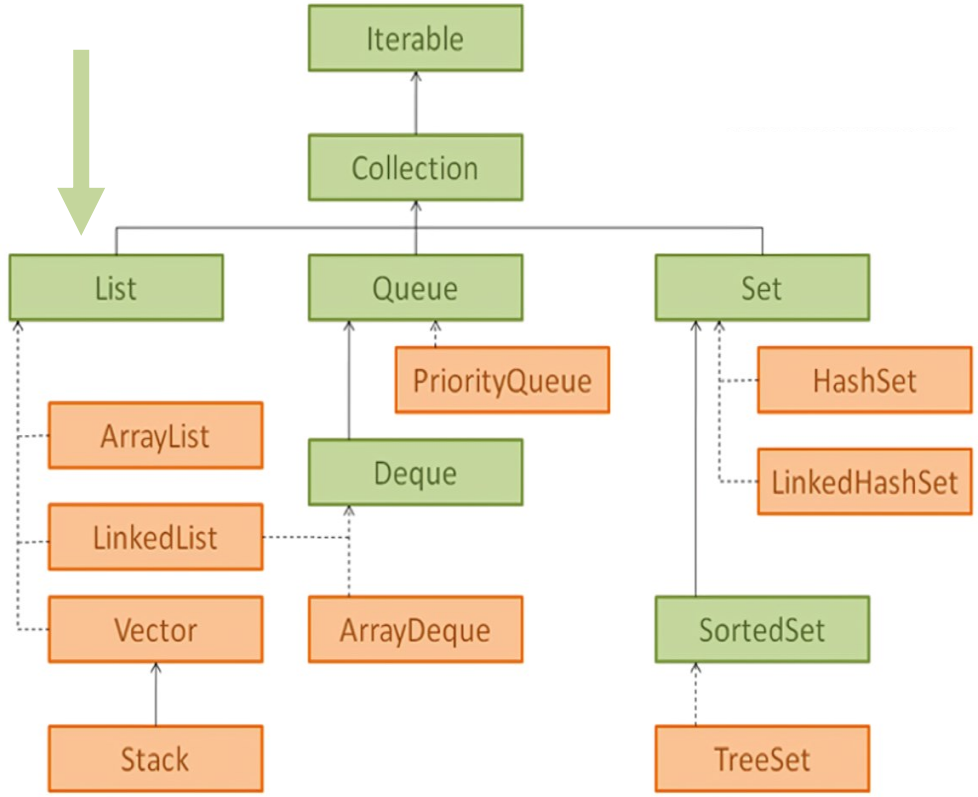
Queue Interface

- Override the `add()` behavior (add to the tail)
- Add new behavior `peek()` and `poll()` behavior to get the head of queue



`List` Interface

```
public interface List<E> extends Collection<E> {
    E get(int index);
    E set(int index, E element);     // Optional
    void add(int index, E element);  // Optional
    Object remove(int index);        // Optional
    boolean addAll(int index, Collection<? extends E> c);
                                     // Optional
    int indexOf(Object o);
    int lastIndexOf(Object o);

    List<E> subList(int from, int to);

    ListIterator<E> listIterator();
    ListIterator<E> listIterator(int index);
}
```



New behaviors of List: index

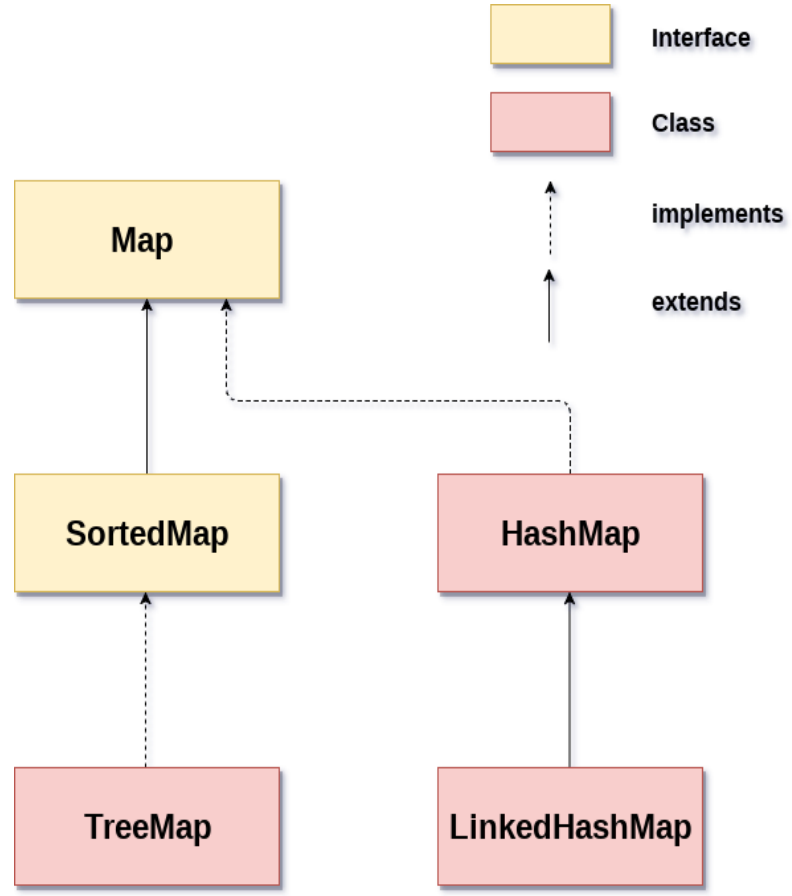
List Idioms

```
List<Type> a, b;

// Concatenate two lists
a.addAll(b);
```

Common behaviors (containsAll, addAll, retainAll, removeAll) from Collections!

Map Interface

```
public interface Map<K,V> {
    int size();
    boolean isEmpty();
    boolean containsKey(Object key);
    boolean containsValue(Object value);
      V    get(Object key);
      V    put(K key, V value);   // Optional
      V    remove(Object key);    // Optional
    void putAll(Map<? Extends K, ? Extends V> t);
    void clear();          // Optional
    // Collection Views
    public Set<K> keySet();
    public Collection<V> values();
    public Set<Map.Entry<K,V>> entrySet();
}
```

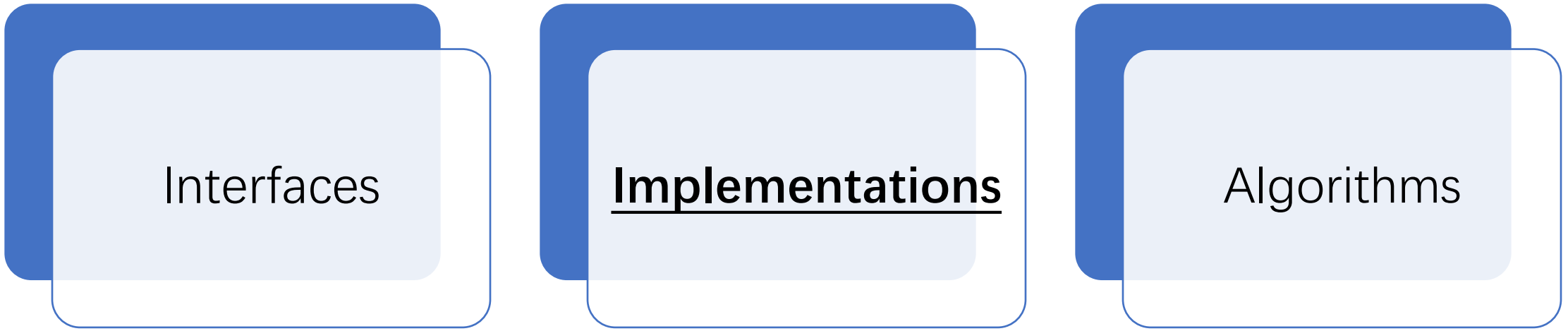


Map Idioms

```
// As of Java 5 (2004)
for (Key k : m.keySet())
    System.out.println(i.next());

// "Map algebra"
Map<Key, Val> a, b;
boolean isSubMap = a.entrySet().containsAll(b.entrySet());
Set<Key> commonKeys =
    new HashSet<>(a.keySet()).retainAll(b.keySet)
//Remove keys from a that have mappings in b
a.keySet().removeAll(b.keySet());
```

Common behaviors (containsAll, addAll, retainAll, removeAll) from Collections!

Core Elements in the Java Collections Framework

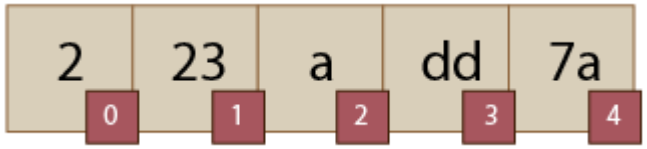


General-purpose Implementations

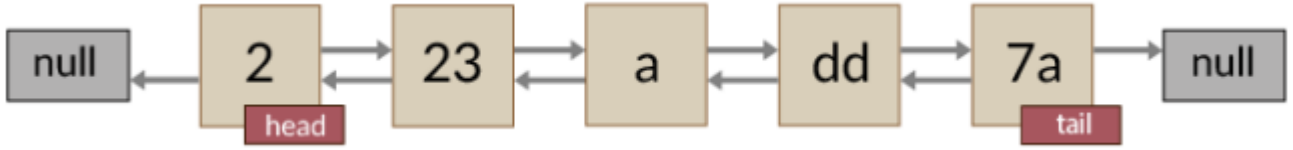
- The Collection framework provides several general-purpose implementations of the `Set`, `List` , and `Map` interfaces
- `HashSet`, `ArrayList`, and `HashMap` are most often used

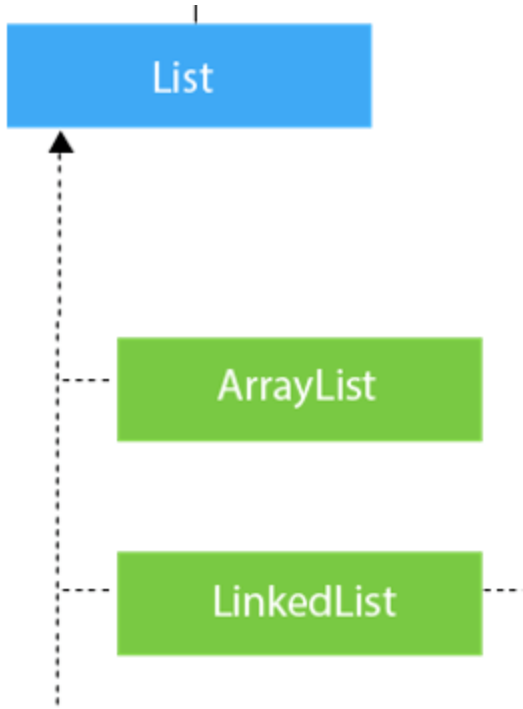
List Implementation

- `ArrayList`: internally uses an array to store the elements



- `LinkedList`: internally uses a doubly linked list to store the elements.

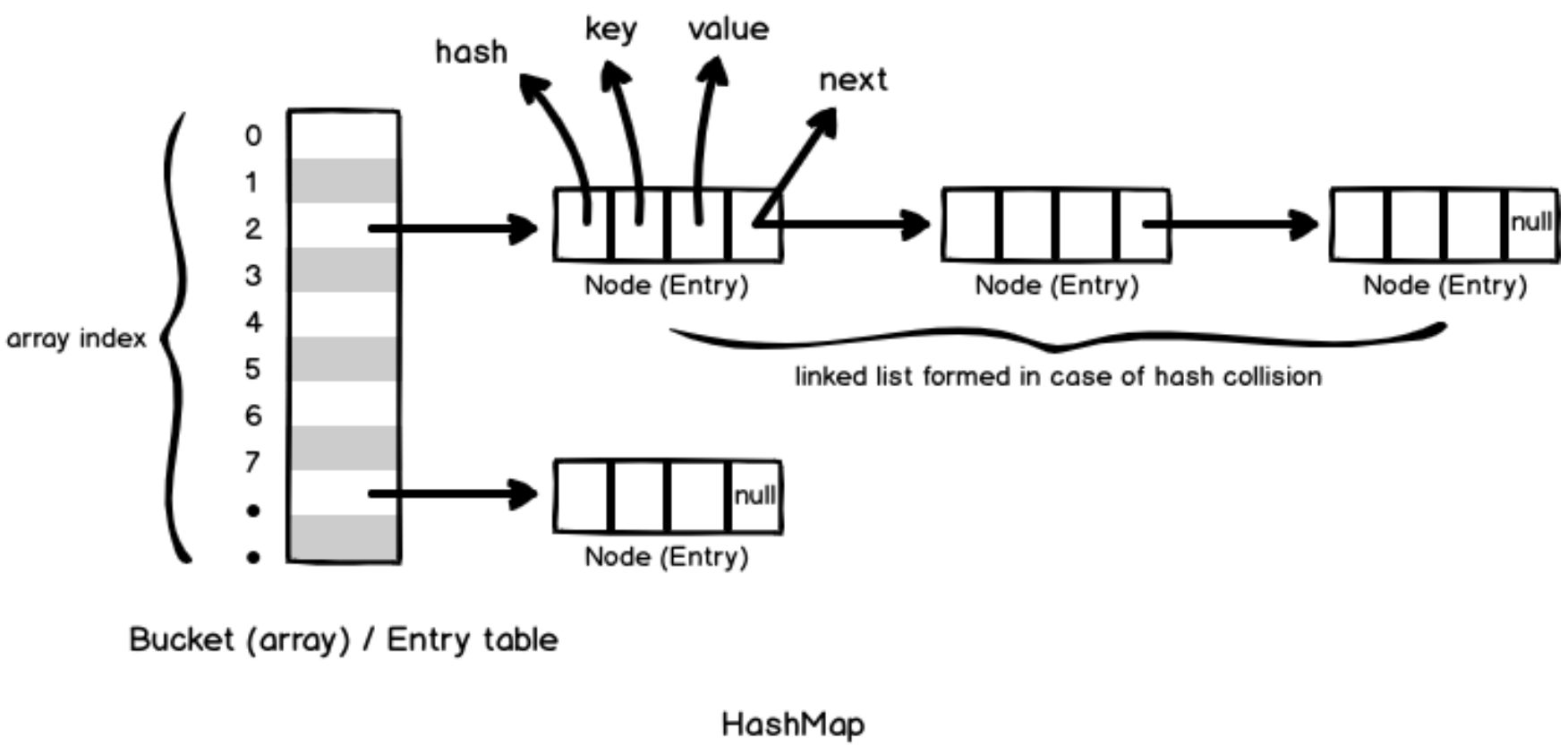


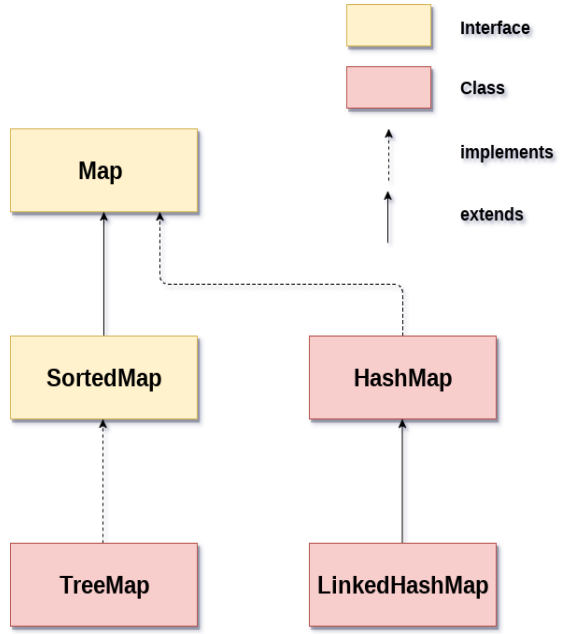


Choosing an Implementation - List

- **`ArrayList`**: Accessing an element takes constant time (O(1))
- **`LinkedList`**: Accessing and element takes O(n) time. LinkedList uses more memory than ArrayList.
- Summary: ArrayList is preferable in many more use-cases than LinkedList. If you're not sure — just start with **ArrayList**.

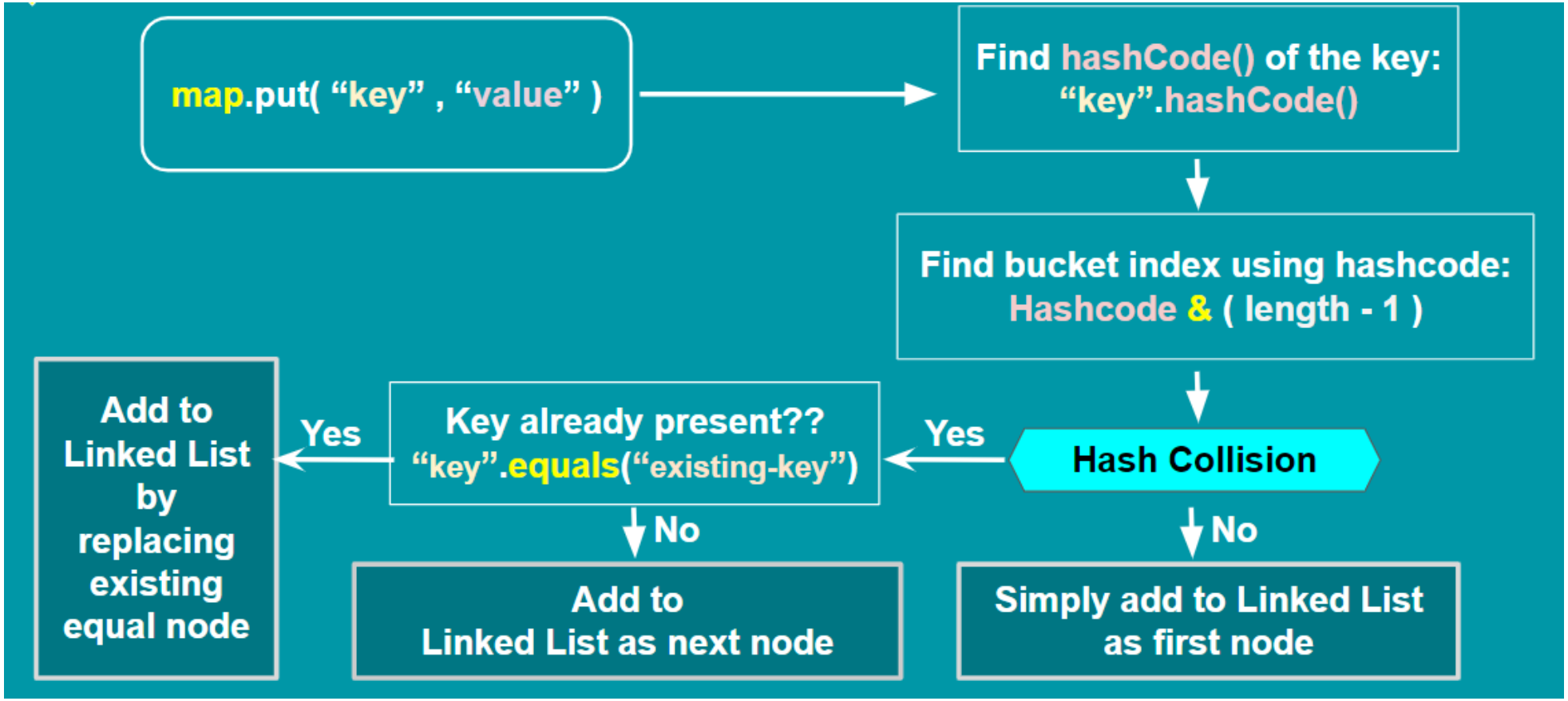
Map Implementation - HashMap





Source: https://www.javaquery.com/2019/11/how-hashmap-works-internally-in-java.html

Map Implementation - HashMap



Source: https://docs.google.com/presentation/d/1jEIOUz-FTG3Ea9FqxDQEiyTTCZGj7zhRMCOgYx2g9dM/edit#slide=id.g94208dd8e0_0_46

Map Implementation - HashMap

```
map.put("FB", 1);
map.put("LD", 2);
map.put("Ea", 3);
map.put("FB", 4);
```

```
hashcode of FB = 2236  | index 12
hashcode of LD = 2424  | index  8
hashcode of Ea = 2236  | index 12
```

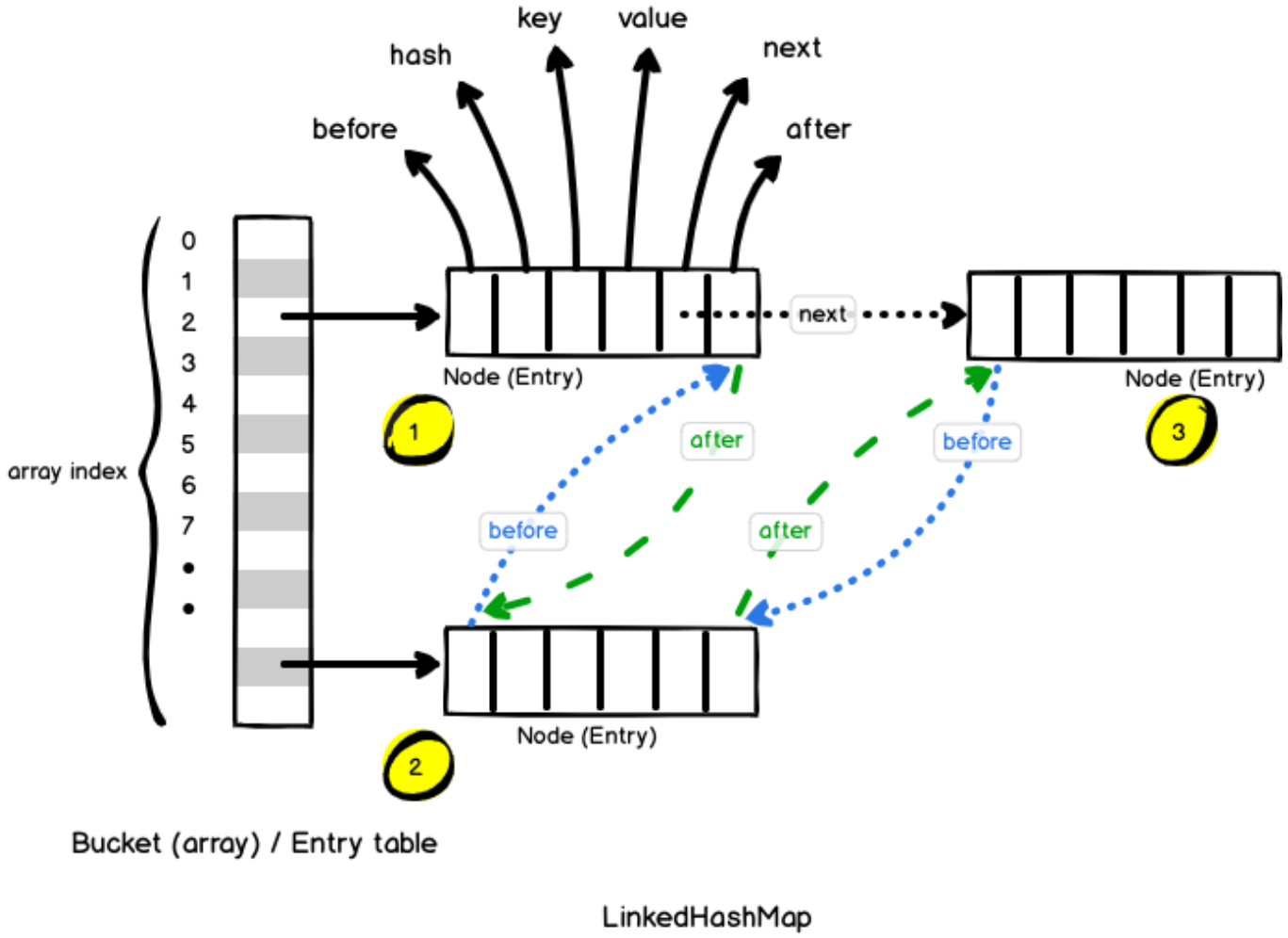
What does the internal HashMap look like?

Overriding `hashCode` and `equals`

- `hashCode()` returns an integer value. By default, it converts the internal address of the object into an integer
- `equals()` checks if objects are equal. By default, `Object.equals(Object obj) { return (this == obj);}`
- If two objects are equal according to the `equals(Object)` method, then calling the `hashCode` method on each of the two objects must produce the same integer result (if you override `equals`, you must override `hashCode`.).

Map Implementation - LinkedHashMap

LinkedHashMap uses before and after to preserve the **insertion order** of the keys

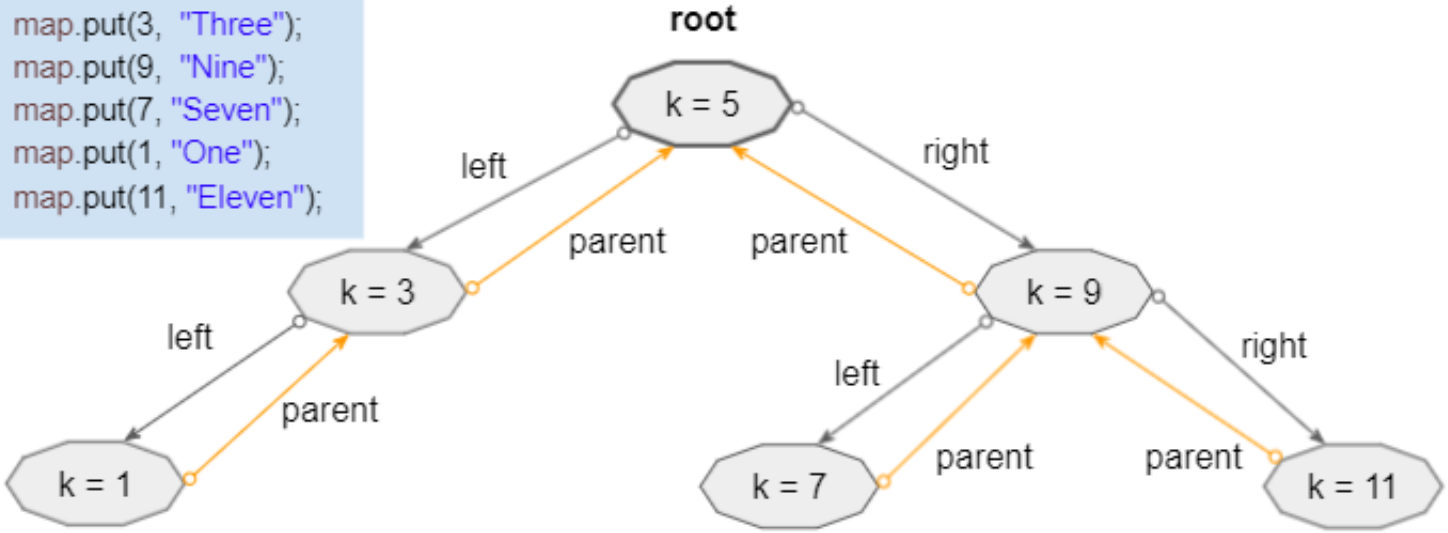


Source: https://www.javaquery.com/2019/12/how-linkedhashmap-works-internally-in.html

Map Implementation - TreeMap

- Use TreeMap when keys **need to be ordered** using their natural ordering or by a Comparator.

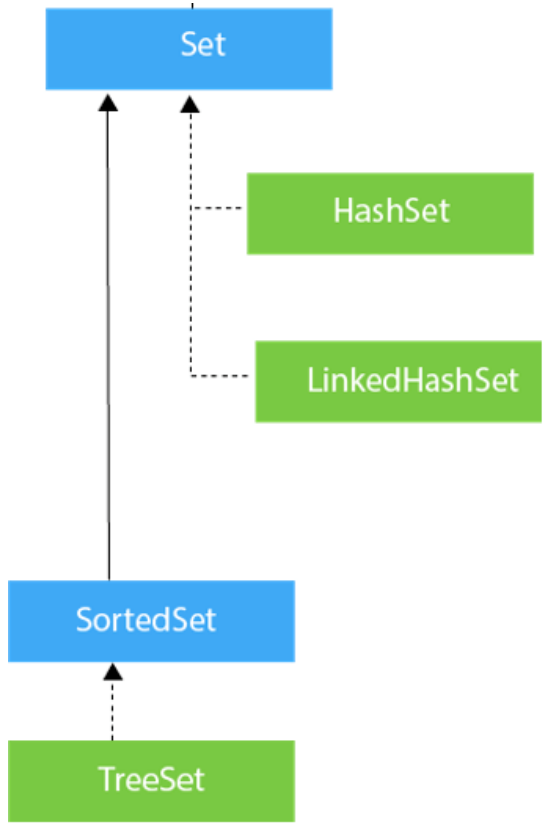
```
map.put(5, "Five");
map.put(3,  "Three");
map.put(9,  "Nine");
map.put(7, "Seven");
map.put(1, "One");
map.put(11, "Eleven");
```



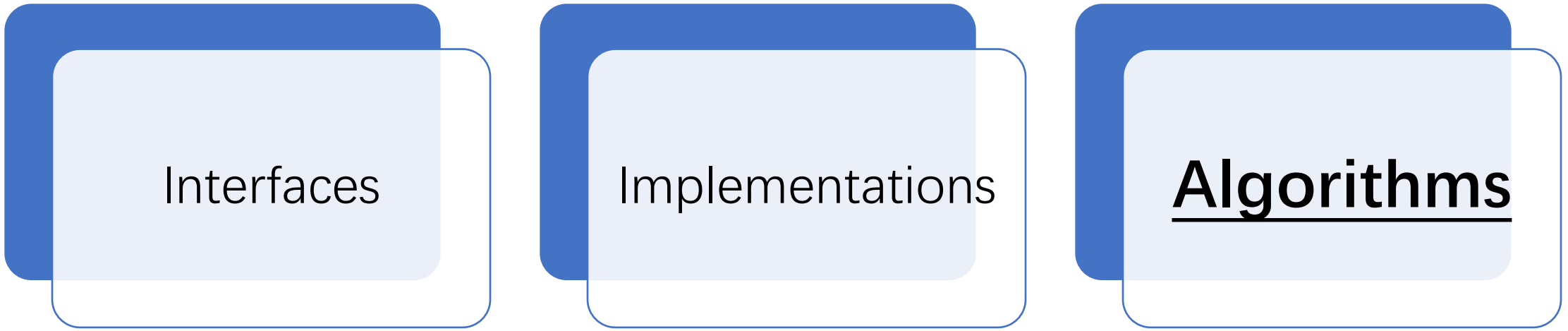
Source: https://o7planning.org/13597/java-treemap

Choosing an Implementation – Set

HashSet	LinkedHashSet	TreeSet
HashSet internally uses HashMap to store its elements.	LinkedHashSet internally uses LinkedHashMap to store its elements.	TreeSet internally uses TreeMap to store its elements.
HashSet doesn't maintain any order of elements.	LinkedHashSet maintain insertion order of elements.	TreeSet maintains default natural sorting order.
HashSet gives better performance than LinkedHashSet and TreeSet.	The performance of LinkedHashSet is between HashSet and TreeSet.	The TreeSet gives less performance than HashSet and LinkedHashSet.
HashSet allow maximum one null element.	LinkedHashSet also allow maximum one null element.	The TreeSet doesn't allow even single null element.



Core Elements in the Java Collections Framework



Reusable Algorithms

`Collections` class in java represents an utility class in java.util package.
It contains exclusively static methods that operate on or return collections
Most methods are generic.

```
static <T extends Comparable<? super T>> void sort(List<T> list);
static int binarySearch(List list, Object key);
static <T extends Comparable<? super T>> T min(Collection<T> coll);
static <T extends Comparable<? super T>> T max(Collection<T> coll);
static <E> void fill(List<E> list, E e);
static <E> void copy(List<E> dest, List<? Extends E> src);
static void reverse(List<?> list);
static void shuffle(List<?> list);
```

java.lang.Object
    java.util.Collections

---

public class **Collections**
extends Object

Why Generic (Reusable) Algorithms?

`max()` is a common algorithm for collections. But we do not want to write, test, debug this max for different types of collections

```
max(ArrayList<T> list)
max(LinkedList<T> list)
```

`Collections.max()` is implemented to take any collection with comparable elements

```
public static <T extends Comparable<? super T>> T max(Collection<? extends T> coll) {
    Iterator<? extends T> i = coll.iterator();
    T candidate = i.next();

    while (i.hasNext()) {
        T next = i.next();
        if (next.compareTo(candidate) > 0)
            candidate = next;
    }
    return candidate;
}
```

Why Generic (Reusable) Algorithms?

```
String[] wordlist = {"This", "is", "CS209A", "Java2"};
List<String> slist = Arrays.asList(wordlist);

Set<Integer> iset = new HashSet<>();
iset.add(3);
iset.add(5);
iset.add(2);

Queue<String> squeue = new LinkedList<>();
squeue.add("Hello");
squeue.add("World");
squeue.add("Java2");

System.out.format("List max: %s%n", Collections.max(slist));
System.out.format("Set max: %s%n", Collections.max(iset));
System.out.format("Queue max: %s%n", Collections.max(squeue));
```

```
List max: is
Set max: 5
Queue max: World
```

Sorting Algorithm

```
public static <T extends Comparable<? super T>> void sort(List<T> list)
```

`sort()` reorders a list according to an ordering relationship

```
List<String> strings;      // Elements type: String
    ...
Collections.sort(strings); // Alphabetical order


LinkedList<Date> dates;    // Elements type: Date
    ...
Collections.sort(dates);   // Chronological order
```

Any restrictions for the elements?

Sorting Algorithm

- `String` and `Date` both implement the `Comparable` interface (`compareTo(T o)`), allowing their objects to be sorted automatically
- `Collections.sort(list)` has compilation error if list elements do not implement `Comparable`

Classes Implementing Comparable

Class	Natural Ordering
`Byte`	Signed numerical
`Character`	Unsigned numerical
`Long`	Signed numerical
`Integer`	Signed numerical
`Short`	Signed numerical
`Double`	Signed numerical
`Float`	Signed numerical
`BigInteger`	Signed numerical
`BigDecimal`	Signed numerical
`Boolean`	`Boolean.FALSE < Boolean.TRUE`
`File`	System-dependent lexicographic on path name
`String`	Lexicographic
`Date`	Chronological

The `Comparator<T>` Interface

```
public interface Comparator<T>
```

- The `Comparable` interface is used to compare objects using one of their property as the default sorting order.
  - Provide `compareTo(T o)`
  - A comparable object can compare itself with another object

- The `Comparator` interface is used to compare two objects of the same class by different properties
  - Provide `compare(T o1, T o2)`
  - Comparator is a separate class and external to the element type being compared

Sorting Algorithm

```java
public class Employee implements Comparable<Employee>{
    String name;
    int id;
    int age;

    @Override
    public int compareTo(Employee e) {
        return name.compareTo(e.name);
    }
```

Default ordering is by name

```java
public class EmployeeIdComparator implements Comparator<Employee>{

   public int compare(Employee o1, Employee o2) {
      if (o1.getId() < o2.getId()) {
         return -1;
      } else if (o1.getId() > o2.getId()) {
         return 1;
      } else {
         return 0;
      }
   }
}
```

```java
public class EmployeeAgeComparator implements Comparator<Employee>{

   public int compare(Employee o1, Employee o2) {
      if (o1.getAge() < o2.getAge()) {
         return -1;
      } else if (o1.getAge() > o2.getAge()) {
         return 1;
      } else {
         return 0;
      }
    }
}
```

TAO Yida@SUSTECH

Sorting Algorithm

```
List<Employee> employees = new ArrayList<>();

employees.add(new Employee("Bob", 1, 20));
employees.add(new Employee("Alice", 4, 22));
employees.add(new Employee("Dave", 2, 21));
employees.add(new Employee("Carol", 3, 25));

//Sorted by natural order (alphabetical order of name)
Collections.sort(employees);
System.out.println(employees);

//Sorted by id
Collections.sort(employees, new EmployeeIdComparator());
System.out.println(employees);

//Sorted by age
Collections.sort(employees, new EmployeeAgeComparator());
System.out.println(employees);
```

```
[Id: 4, age: 22, name: Alice ],
[Id: 1, age: 20, name: Bob ],
[Id: 3, age: 25, name: Carol ],
[Id: 2, age: 21, name: Dave ]]

[Id: 1, age: 20, name: Bob ],
[Id: 2, age: 21, name: Dave ],
[Id: 3, age: 25, name: Carol ],
[Id: 4, age: 22, name: Alice ]]

[Id: 1, age: 20, name: Bob ],
[Id: 2, age: 21, name: Dave ],
[Id: 4, age: 22, name: Alice ],
[Id: 3, age: 25, name: Carol ]]
```

Further Reading

ORACLE Java Documentation

The Java™ Tutorials

« Previous

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Trail: Collections: Table of Contents

https://docs.oracle.com/javase/tutorial/collections/TOC.html

Evolution of Java Collections

Release, Year	Changes
JDK 1.0, 1996	Java Released: `Vector`, `Hashtable`, `Enumeration`
JDK 1.1, 1996	(No API changes)
J2SE 1.2, 1998	Collections framework added
J2SE 1.3, 2000	(No API changes)
J2SE 1.4, 2002	`LinkedHash{Map,Set}`, `IdentityHashSet`, 6 new algorithms
J2SE 5.0, 2004	Generics, for-each, enums: generified everything, `Iterable` `Queue`, `Enum{Set,Map}`, concurrent collections
Java 6, 2006	`Deque`, `Navigable{Set,Map}`, `newSetFromMap`, `asLifoQueue`
Java 7, 2011	No API changes. Improved sorts & defensive hashing
Java 8, 2014	Lambdas (+ streams and internal iterators)

https://www.cs.cmu.edu/~charlie/courses/15-214/2016-fall/slides/15-collections%20design.pdf

Next Lecture

- Functional Programming
- Lambda Expressions